

EXACT PROJECTION QUALITY OF ONETWO SOBOLOV SEQUENCES AT 65,536 POINTS

LENNART RUDOLPH 

ABSTRACT. Low-dimensional projection quality is a practical bottleneck for Sobol’ sequences used in quasi-Monte Carlo integration and computer graphics. For a digital net with $2^{16} = 65,536$ points, the t -value measures exact equidistribution in dyadic boxes, with smaller values indicating stronger uniformity. We determine the exact $m = 16$ census for all 345 pair-aligned four-dimensional blocks in the released 692-dimensional OneTwo construction: 9 blocks have $t = 4$, and the remaining 336 have $t = 5$. For dimensions 25–28, determinant-one minors prove the upper bound, while a unique row dependency with mask `0xD2F` yields an explicit empty dyadic box for the lower bound. Exact $\mathbb{F}_2$ computations agree across three independent rank routes, direct enumeration checks all 65,536 points, and a Lean companion checks the finite worked-block encoding. The worked certificate is self-contained; reproducing the complete census additionally uses the exact upstream direction-number table identified by pinned commit and file hashes.

1. INTRODUCTION AND SOURCE BOUNDARY

Sobol’ sequences are among the standard digital constructions used in quasi-Monte Carlo integration and computer graphics [3, 7, 10]. Their t -value measures exact equidistribution in elementary intervals: smaller is better, and $t = 0$ is ideal. Because the same high-dimensional sequence often drives many sampling decisions, the quality of low-dimensional projections is as important as that of the full point set. This has motivated both better direction-number constructions [4] and efficient algorithms for computing projection t -values [6, 8]. Projection quality has long been studied as a separate practical issue for digital nets and high-dimensional low-discrepancy sequences [9, 11]. Those works develop projection-oriented quality measures and comparisons; the contribution here is a source-pinned exact census and auditable finite certificate for one released construction. Recent work also uses small projected t -values as sufficient conditions for recovering low-dimensional cubature grids inside transformed digital nets [12]; that application is complementary to the source-table certificate proved here.

Date: August 14, 2026.

2020 Mathematics Subject Classification. 11K36, 11K38, 65C05, 68V05.

Key words and phrases. Sobol’ sequence, digital net, t -value, quasi-Monte Carlo, exact census, finite-field rank.

Bonneel, Coeurjolly, Iehl, and Ostromoukhov recently introduced the *OneTwo* construction [1]. Their released table has 692 dimensions [1, Sec. 5.1]. We call the disjoint pairs $(2k - 1, 2k)$, $1 \leq k \leq 346$, the *protected pairs*; [1, Fig. 8] states the $(1, 2)$ property for these pairs. For each pair-aligned block formed from two successive protected pairs, Section 5.2 describes numerical acceptance targets $t \leq 3$ through $m \leq 10$ and $t \leq 4$ through $m \leq 15$ [1, Sec. 5.2]. These are construction criteria rather than an all-depth theorem. Our full-table audit confirms the latter target at $m = 15$, but not the literal all-window reading of the former; the precise diagnostic is reported below, and the $m \leq 10$ language is not used as a premise. The abstract’s phrase “all consecutive pairs” is read in this protected-pair sense; the unaligned pair $(26, 27)$ below has $t = 5$ at $m = 16$.

This note determines the exact value at $m = 16$ for every pair-aligned block in the released table. Dimensions are numbered from one in theorem statements and from zero in the C++ array. The pair-aligned windows are

$$W_k = (2k - 1, 2k, 2k + 1, 2k + 2), \quad 1 \leq k \leq 345.$$

Theorem 1.1. *Take the point set $\{x_n : 0 \leq n < 2^{16}\}$ obtained from the *OneTwo* file `sampler/onetwo_matrices.cpp` at repository commit*

163127a0c1731ffe2d43cd685067795c2689a308.

using binary input indices as defined in Section 2. For each W_k , use its first sixteen direction columns and no scrambling. The exact four-dimensional digital-net value is $t = 4$ precisely when the first dimension of W_k belongs to

$$\{1, 7, 31, 49, 67, 195, 225, 243, 373\},$$

and is $t = 5$ for the other 336 windows. In particular, dimensions 25–28 have exact value $t = 5$. Blob and complete-file digests appear in Table 2.

Table 1 is the census in compact form. The $t \leq 4$ behavior extends to $m = 16$ in exactly nine windows; the same $t \leq 4$ statement cannot be extended to $m = 16$ for the other 336.

TABLE 1. Exact $m = 16$ values for all pair-aligned windows W_k .

Exact t	Number of windows	First dimensions of the windows
4	9	1, 7, 31, 49, 67, 195, 225, 243, 373
5	336	all other odd starts in $\{1, 3, \dots, 689\}$

The complete census is recomputed from the authenticated 692-by-52 source table by a primary evaluator with two different exact-rank kernels. The two kernels share the authenticated parser, composition generator, and census control flow; agreement between them isolates rank reduction but is not a fully independent reconstruction. For each window, the evaluator

starts at $t_0 = 0$ and returns the least t_0 for which every required composition matrix has full row rank. Its machine-readable record also gives a deficient composition, rank, and dependency mask at the immediately stronger boundary. Section 4 gives the exhaustive proof.

The dimensions 25–28 case has two additionally readable halves. An upper certificate proves $t \leq 5$ by checking all 364 required composition matrices. A lower certificate proves $t \not\leq 4$ by a displayed dependency and an empty dyadic box. The full minor list, direct occupancy enumeration, and Lean companion make this representative case independently inspectable without trusting only the census summary.

The certificate files are explicit witnesses rather than records of a numerical search: each upper case names a determinant-one minor, while the lower case names both a row-dependency vector and an inconsistent prefix system. The programs regenerate and check those witnesses using integer and $\mathbb{F}_2$ arithmetic. The theorem therefore does not depend on floating-point rank decisions or on trusting a single enumeration implementation.

The conclusion remains source-specific. It says nothing about optimal four-dimensional direction numbers, other table versions, scrambled point sets, non-pair-aligned windows, or non-power-of-two prefixes.

For a quantitative interpretation, the standard star-discrepancy estimate for a base-two $(t, m, 4)$ -net [3, Corollary 5.3] (see also [7, Theorems 4.5–4.6]) gives

$$D_{2^{16}}^* \leq \frac{2^t}{2^{16}} \sum_{i=0}^3 \binom{16-t}{i}.$$

This upper bound is 299/4096 at $t = 4$ and 29/256 at $t = 5$, a factor 464/299 larger in the latter case. These are guaranteed bounds, not claims that the actual discrepancies attain them.

2. DIGITAL-NET CONVENTIONS AND THE RANK CRITERION

We fix conventions before giving any machine data. Let $C_1, \dots, C_s \in \mathbb{F}_2^{m \times m}$. For $n = \sum_{c=0}^{m-1} n_c 2^c$, write $\mathbf{n} = (n_0, \dots, n_{m-1})^\top$. The output digits in coordinate i are

$$(y_{i,1}, \dots, y_{i,m})^\top = C_i \mathbf{n}, \quad x_{n,i} = \sum_{r=1}^m y_{i,r} 2^{-r}.$$

This is the usual base-two digital point set of size 2^m . The released sampler emits 32-bit coordinates, whereas these matrices retain the first 16 output digits. Every elementary-interval condition relevant to an $m = 16$ net has each side exponent $d_i \leq 16$; membership therefore depends only on those first 16 digits. Consequently the top-left 16×16 blocks determine the full $m = 16$ t -value of the released 32-bit points.

Definition 2.1. For a point set P of 2^m points in $[0, 1)^s$, its t -value is

$$t(P) = \min\{t_0 \in \{0, \dots, m\} : P \text{ is a } (t_0, m, s)\text{-net in base 2}\}.$$

The set is nonempty because the $t_0 = m$ condition imposes only the zero-depth composition.

We use $\mathbb{N} = \{0, 1, 2, \dots\}$. For $d = (d_1, \dots, d_s) \in \mathbb{N}^s$, let $C(d)$ be the matrix obtained by stacking, in coordinate order, the first d_i rows of C_i . Its row count is $|d| = \sum_i d_i$. We use the standard rank characterization of digital nets [3, Theorem 4.52]; see also [7, Chapter 4] and [6, Section 2].

Proposition 2.2 (Digital-net rank criterion). *Let $0 \leq t_0 \leq m$. The point set generated by $C_1, \dots, C_s$ is a (t_0, m, s) -net in base two if and only if*

$$\text{rank}_{\mathbb{F}_2} C(d) = m - t_0$$

for every weak s -composition d of $m - t_0$.

Proof. Fix a composition d of $m - t_0$. The linear map

$$L_d : \mathbb{F}_2^m \longrightarrow \mathbb{F}_2^{m-t_0}, \quad \mathbf{n} \longmapsto C(d)\mathbf{n},$$

assigns to an input index exactly the prescribed leading output digits in the corresponding elementary interval. If $C(d)$ has full row rank, then L_d is surjective and every fiber has cardinality $2^{m-(m-t_0)} = 2^{t_0}$. Thus every interval of that shape contains exactly 2^{t_0} points. Conversely, if $C(d)$ is rank-deficient, L_d misses some prefix vector, so the corresponding interval of volume 2^{t_0-m} is empty. Applying this argument to every weak composition gives both directions of the criterion. $\square$

Lemma 2.3 (Monotonicity in t_0). *If the rank condition holds for t_0 , then it holds for every $t'_0 \geq t_0$ with $t'_0 \leq m$.*

Proof. For a weak composition d of $m - t'_0$, extend it coordinatewise to a weak composition d' of $m - t_0$. The rows of $C(d)$ form a subset of the independent rows of $C(d')$, so $C(d)$ has full row rank. $\square$

Lemma 2.4 (Coordinate projection). *Every coordinate projection of a (t_0, m, s) -net is a (t_0, m, r) -net on the retained r coordinates.*

Proof. Extend each weak r -composition of $m - t_0$ by zeros in the omitted coordinates and apply Proposition 2.2. $\square$

Here a (t_0, m, s) -net has exactly 2^{t_0} points in every elementary interval of volume 2^{t_0-m} . Proposition 2.2 is the linear-algebraic form of the same assertion: a choice of d_i prefix bits gives $|d|$ affine equations in the m input digits. Full row rank makes every right-hand side consistent with exactly $2^{m-|d|}$ solutions.

For $m = 16$, the desired upper bound $t \leq 5$ requires full row rank at total depth 11. The stronger bound $t \leq 4$ would require full row rank at total depth 12. Thus the two finite certificates below establish, respectively, the net property at $t_0 = 5$ and its failure at $t_0 = 4$. The final proof of Theorem 1.1 also checks every $t_0 < 4$ directly as a redundant confirmation of Lemma 2.3. There are $\binom{11+4-1}{4-1} = \binom{14}{3} = 364$ total-11 cases.

3. PINNED SOURCE AND MATRIX TRANSCRIPTION

The theorem is about released data, so source fidelity is part of the mathematics. Table 2 records the complete pin. The Git blob identifier was recomputed using the canonical `blob <length>\0<contents>` serialization, not copied from a web page.

TABLE 2. Pinned source identity and indexing convention.

Field	Value
Repository	<code>liris-origami/OneTwoSobolSequences</code>
Commit	<code>163127a0c1731ffe2d43cd685067795c2689a308</code>
Path	<code>sampler/onetwo_matrices.cpp</code>
Git blob SHA-1	<code>f183f3d97496ee3a60a0330f199ffc7a3dcf8f7f</code>
Complete-file SHA-256	<code>6f865af6a63983188e1ef8a03adaaa54</code> <code>fc9826ef98f9d4ccda649b1683566e51</code>
Complete array shape	692×52 32-bit integers
Zero-based rows	24, 25, 26, 27
One-based dimensions	25, 26, 27, 28
Relevant prefix	first 16 direction columns
Prefix SHA-256	<code>6543abe1fe91c5a7b4f2e07bf3a3f94a</code> <code>70f81ddc6b17dab5285ea9d9c6147ce2</code>

The source declaration is

```
std::array<unsigned, 52> onetwo_matrices[692].
```

Each row contains 52 input-index direction columns represented as unsigned 32-bit output integers. Columns 33–52 continue the input-index sequence while retaining 32 bits of coordinate precision; only columns 1–16 can affect indices $0 \leq n < 2^{16}$, so the theorem uses exactly those columns. The repository, commit, and access date are cited separately as [5].

The prefix digest has a fixed serialization: traverse dimensions 25, 26, 27, and 28 in that order; within each dimension traverse direction columns 1 through 16; encode each value as an unsigned 32-bit big-endian integer; and concatenate the resulting 256 bytes without separators. The digest is the SHA-256 of exactly that byte string. This definition makes the printed hash recomputable independently of JSON whitespace or host byte order.

Let $v_{i,c}$ be direction integer c for coordinate i , with $0 \leq c < 16$. The bit at position $31 - r$ becomes the coefficient of input digit c in output row r :

$$(1) \quad C_i[r, c] = \text{bit}_{31-r}(v_{i,c}), \quad 0 \leq r, c < 16.$$

For compactness, a row is written as the 16-bit integer $\sum_c C_i[r, c]2^c$. Equation (1) gives the four row blocks in the artifact. The rows used by the lower witness are displayed in the next section; all 64 rows are checked by both Python and Lean.

Two extraction routes were compared. The primary parser reads the entire C++ initializer, requires 692 blocks of length 52, authenticates both full file hashes, and then slices rows 24–27. A reproducible cross-check starts from a frozen 4-by-16 excerpt and authenticates its serialized prefix digest; an older supplied-verifier result is retained only as an archived transcript. Both reproducible routes obtain the same 64 directions and transposed rows. This directly tests the most dangerous interpretation errors: table drift, one-based versus zero-based dimensions, output-bit orientation, and window location. A common permutation of input columns merely relabels the input bits and cannot change the point set; likewise, permuting the four coordinates cannot change a t -value.

The published guarantees supply an independent convention check, separate from the hash chain. Exact rank computation from the frozen prefix gives the four-dimensional ladder in Table 3.

TABLE 3. Exact t -value for dimensions 25–28 using the first 2^m points, $m = 1, \dots, 16$.

m	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
t	0	1	2	3	2	2	3	3	3	3	4	4	4	4	4	5

Thus $t \leq 3$ through $m = 10$ and $t \leq 4$ through $m = 15$, exactly as published for this pair-aligned block. At $m = 16$, the three adjacent two-dimensional windows (25, 26), (26, 27), and (27, 28) have values 1, 5, 1, respectively. This signature confirms that rows 24–27 are aligned as two protected pairs; the middle, unaligned pair is not covered by the pair guarantee. As a falsification control, reversing the complete 16-row output block once and then taking its first m rows gives the ladder 1, 2, $\dots$, 15, 15, violating the published bounds. Input-column reversal and coordinate reversal are invariance checks and both leave the certified $m = 16$ value equal to five.

The complete authenticated table provides a stricter source-convention audit. All 346 protected pairs satisfy $t \leq 1$ through $m = 16$, and all 345 pair-aligned four-windows satisfy $t \leq 4$ at $m = 15$ (the exact distribution is $t = 3$ once and $t = 4$ in 344 cases). In contrast, a literal evaluation of the printed $t \leq 3$ through $m \leq 10$ target gives, already at $m = 10$, 92 windows with $t = 3$ and 253 with $t = 4$. Likewise, interpreting the moving six-dimensional block as the last two protected pairs plus the new pair produces 343 pair-depth violations of the printed all-pairs $t \leq 3$ target across 195 of 344 blocks. Both exact-rank kernels agree on these diagnostics. They may reflect an unstated construction-stage ordering or a difference between acceptance data and the released C++ order; this note does not try to resolve that provenance question and does not rely on either disputed criterion. The verified $m = 15$ bound and the new $m = 16$ census are statements about the pinned released array itself.

For completeness, Table 4 records the entire exact two-dimensional ladder for each adjacent pair. The nonmonotonicity in m is normal: these are values for different finite point sets, not successive bounds on one fixed net.

TABLE 4. Exact adjacent-pair t -values for the first 2^m points.

Pair \ m	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
25–26	0	0	1	1	1	1	0	1	1	1	1	1	1	1	1	1
26–27	0	0	1	1	2	2	3	3	2	2	3	1	2	3	4	5
27–28	0	0	1	1	1	1	1	0	1	1	1	1	1	1	1	1

4. EXHAUSTIVE PROOF OF THE 345-WINDOW CENSUS

Proof of the census assertion in Theorem 1.1. The complete authenticated table contains 692 direction rows. The verifier uses each zero-based start

$$j \in \{0, 2, \dots, 688\}$$

and transposes the first sixteen direction integers of rows $j, j+1, j+2, j+3$ according to (1). This is exactly the one-based window $(j+1, j+2, j+3, j+4)$, so there are 345 cases and no endpoint truncation.

For a fixed window, the evaluator tries $t_0 = 0, 1, \dots, 16$. At each t_0 , it enumerates every ordered weak four-composition of $16 - t_0$, stacks the prescribed leading rows, and tests equality of the rank with $16 - t_0$. It returns the first successful t_0 . By Proposition 2.2, that returned integer is the exact t -value; no monotonicity shortcut is used. One implementation performs left-to-right Gaussian elimination over packed binary rows. A second rank kernel in the same evaluator inserts rows into a highest-set-bit pivot map. The two kernels agree window-by-window on all 345 outputs. A separately written set-valued cross-check represents rows as sets of column positions, generates compositions recursively, and eliminates by smallest-column set symmetric difference; it returns the same distribution and exceptional starts. This cross-check shares the authenticated source format and the mathematical rank criterion with the primary audit, but not its packed-row representation, composition generator, or elimination code.

The release file `census.json`, whose SHA-256 is

28fc79b3c942e157d3ea66f7d077de1d84e3898282bb89342020b458b816bf41,

records the source identity, indexing rule, exact value, upper row total and composition count, and a lower-bound witness for every window. Independently of the early-exit exact evaluator, the verifier enumerates all deficient compositions at the immediately stronger boundary for every window and checks each deficiency with both rank algorithms. It then requires the distribution and the nine exceptional starts printed in Table 1. Hence Theorem 1.1’s census follows from the rank criterion and an exhaustive finite calculation over $\mathbb{F}_2$. $\square$

5. THE COMPLETE UPPER CERTIFICATE

Theorem 5.1. *For one-based dimensions 25–28 and every weak four-composition d of 11, the matrix $C(d)$ has rank 11 over $\mathbb{F}_2$. Hence that projection satisfies $t \leq 5$.*

Proof. The certificate file `upper-minors.json` contains 364 records, one for each ordered weak composition of 11. A record consists of d , eleven zero-based column indices, the corresponding one-based indices, and the value one for the determinant modulo two. For each record, the verifier reconstructs $C(d)$ from the pinned directions, restricts to the recorded columns, and performs exact elimination over $\mathbb{F}_2$. Every restricted 11×11 matrix has determinant one. Thus each $C(d)$ contains a nonsingular square minor and has row rank 11.

The composition generator produces 364 distinct tuples, every tuple has nonnegative entries and total 11, and the count equals $\binom{14}{3}$. These records exhaust the cases required by Proposition 2.2; hence $t \leq 5$. $\square$

The primary verifier finds the lexicographically first invertible column set by testing combinations and uses left-to-right Gaussian elimination. A frozen transcript from the supplied verifier uses a different pivot dictionary and a greedy independent-column selector. The two emitted lists agree record-for-record. The canonical certificate JSON has SHA-256

399a301dd28c7ab0531ab6c311876b3adf75bb7e0b6d8365144629a1ebb40040.

The file is read back after emission and all determinants are checked again, so serialization is included in the audit.

There is also a geometric cross-check. Direct point enumeration computes the prefix key of every one of the 65,536 points for every total-11 composition. Every elementary interval in every one of the 364 shape families contains exactly $2^{16-11} = 32$ points. This repeats the upper conclusion without calling the minor-selection routine.

6. THE LOWER RANK AND EMPTY-BOX CERTIFICATES

Let

$$d_* = (0, 3, 3, 6).$$

The twelve rows of $C(d_*)$, in stacked order, are

$$(2) \quad \begin{array}{l} \text{AF05, C566, 9664;} \\ \text{86A7, D08A, 3444;} \\ \text{BFF5, E9BA, B354, E718, 8750, 7AE0.} \end{array}$$

The semicolons mark the boundaries between dimensions 26, 27, and 28. All row constants in this section are hexadecimal 16-bit integers under the packing convention of equation (1).

Theorem 6.1. *The matrix $C(d_*)$ has rank 11, and*

$$\begin{aligned} &\text{AF05 xor C566 xor 9664 xor 86A7} \\ &\text{xor 3444 xor B354 xor 8750 xor 7AE0} = 0. \end{aligned}$$

This is the unique nonempty zero-XOR subset of the twelve rows. Consequently $t \not\leq 4$.

Proof. Exact elimination on the twelve rows in (2) gives rank 11. Reading subset bits from least significant upward, the dependency mask 0xD2F has support

$$\{0, 1, 2, 3, 5, 8, 10, 11\}.$$

The corresponding eight rows are precisely those in the displayed XOR, and direct bitwise evaluation gives zero. This is a nonempty dependence in a total-12 composition matrix. Proposition 2.2 therefore rules out $t \leq 4$. Because $C(d_*)$ has twelve rows and rank 11, its left nullspace—the space of row dependencies—is one-dimensional. Over $\mathbb{F}_2$ it therefore has a unique nonzero element; exhaustive subset XOR recovers exactly 0xD2F. $\square$

Remark 6.2 (Boundary census and pair-local witness). Exactly six of the 455 total-12 compositions are deficient, each with rank 11:

$$\begin{aligned} &(0, 3, 3, 6), (0, 3, 9, 0), (0, 6, 0, 6), \\ &(1, 1, 10, 0), (1, 7, 0, 4), (2, 3, 5, 2). \end{aligned}$$

The second lies entirely in the unaligned middle pair. Its unique dependency has mask 0x924 and the four-row form

$$0x9664 \text{ xor } 0x3444 \text{ xor } 0x7B20 \text{ xor } 0xD900 = 0.$$

Here 0x7B20 and 0xD900 are rows 6 and 9 of dimension 27 in `source-prefix.json`. The corresponding augmented system proves that

$$[0, 1) \times [1/8, 1/4) \times [0, 1/512) \times [0, 1)$$

is empty. Thus the $m = 16$ failure is already inherited from the one adjacent pair inside the block that is not protected as a constructed pair. By Lemma 2.4, its exact two-dimensional value $t = 5$ already forces the four-dimensional lower bound $t \geq 5$; the twelve-row certificate provides a separate explicit four-dimensional obstruction. The verifier checks the complete six-composition census, both unique dependency masks, both augmented ranks, and both empty boxes.

The same dependence yields a human-readable empty box. Prescribe the first three bits in dimension 26 to be 100, the first three in dimension 27 to be 000, and the first six in dimension 28 to be 000000. The resulting elementary interval is

$$(3) \quad B = [0, 1) \times [1/2, 5/8) \times [0, 1/8) \times [0, 1/64).$$

Its twelve prefix equations have coefficient matrix $C(d_*)$. XOR the eight equations selected by 0xD2F. Their left sides cancel by Theorem 6.1, while

their right sides XOR to one: only the first selected equation carries the leading 1 of the prefix 100. The system therefore asserts $0 = 1$ and has no input index. Equivalently, the coefficient matrix has rank 11 and the augmented matrix has rank 12.

Thus B is empty. Its volume is $2^{-(0+3+3+6)} = 2^{-12}$, whereas a $(4, 16, 4)$ -net would place $2^4 = 16$ points in every such interval. Direct enumeration independently finds zero points in B . Across all 4096 boxes of shape d_* , 2048 contain zero points and 2048 contain 32 points, the occupancy pattern predicted by rank 11.

Dimensions 25–28 clause. Theorem 5.1 gives $t \leq 5$ for the detailed block in Theorem 1.1. Conversely, fix $t_0 \leq 4$ and consider the weak composition

$$d' = (4 - t_0, 3, 3, 6)$$

of $16 - t_0$. It dominates $d_* = (0, 3, 3, 6)$ coordinatewise, so $C(d')$ contains all twelve rows of $C(d_*)$. The dependency in Theorem 6.1 persists, and hence $\text{rank } C(d') < 16 - t_0$. Proposition 2.2 shows that the point set is not a $(t_0, 16, 4)$ -net for any $t_0 \leq 4$. Since it is a $(5, 16, 4)$ -net, its exact t -value is five. $\square$

7. ENUMERATION ORDER AND INDEPENDENT CHECKS

Sobol’ implementations are often described using either binary input indices or Gray-code updates [2]. This distinction changes the order of a complete power-of-two block, not its underlying set. The map

$$n \mapsto n \oplus \lfloor n/2 \rfloor$$

is a bijection on the 16-bit strings. The verifier constructs both lists, checks that each contains 65,536 distinct points, and checks set equality. The theorem is therefore insensitive to these two ordering conventions. It would not be legitimate to infer the same conclusion for a truncated or shifted block.

Table 5 summarizes the independent evidence. All arithmetic is integer or $\mathbb{F}_2$ arithmetic; no numerical tolerance is used.

TABLE 5. Exact certificate and cross-check inventory.

Object	Primary certificate route	Cross-check route
Source	full C++ parse, two hashes	frozen-prefix digest
Rows	explicit bit transpose	set-valued transpose
Upper bound	364 ranks and minors	point occupancy
Lower rank	elimination and mask XOR	set-valued elimination
Empty box	augmented rank 12	65,536-point enumeration
Order	binary/Gray set equality	permutation argument
Lean	same frozen rows	independent finite evaluator

The complete-source and prefix hashes test different failure modes. The prefix digest makes the offline theorem input compact, while the Git blob and complete-file digest bind that input to a specific public table. The artifact fails closed if any metadata field, dimension index, direction value, row, minor, dependency bit, rank, or occupancy count changes.

8. LEAN FINITE-CERTIFICATE COMPANION AND TRUST BOUNDARY

The companion is a standalone Lean 4.30.0 project with no Mathlib dependency. It completes four build jobs and isolates a finite native-evaluation trust boundary. Historical working-draft files are retained only in private provenance records and are not part of the public verification claim.

The formal file checks the following finite statements.

- All 64 pinned directions transpose to the 64 stated rows.
- The total-11 and total-12 composition lists have lengths 364 and 455, and every selected stack has the corresponding length. The enumerators through total 16 are also checked to have the advertised coordinate sum.
- The total-11 and total-12 composition lists are duplicate-free.
- Every total-11 selected family is XOR-independent.
- The witness row list has the exact order in (2), and 0xD2F selects a zero-XOR subset.
- The Boolean upper and lower certificates imply the proposition-level conjunction $t \leq 5$ together with $t \not\leq t_0$ for every $t_0 = 0, 1, 2, 3, 4$ in the rank encoding.

The project contains no `sorry`, `admit`, or project-declared axiom. Small structural checks use kernel reduction; the large exhaustive independence computations use `native_decide`. The literal Lean 4.30.0 `#print axioms` transcript identifies the generated native-evaluation dependencies (and `propext` where applicable). This is an explicit compiler-evaluation trust boundary, not a claim of purely kernel-reduced computation.

Nor does the Lean project formalize the full theory of digital nets. It formalizes the finite rank encoding and certificate plumbing for this instance. Proposition 2.2, the identification of direction numbers with digital generator matrices, and the point-set interpretation are mathematical inputs proved in the paper and standard references. The two Python verifiers remain independently inspectable checks of the same finite instance.

9. CONTEXT, LIMITATIONS, AND REPRODUCIBILITY

The census complements, rather than corrects, the OneTwo result. The source construction gives a uniform theorem over its protected pair-aligned blocks in a stated depth range; the present note gives every exact value at the next depth. General t -value algorithms can compute such values [6], but the release here emphasizes a portable record for all 345 windows and a detailed proof object for dimensions 25–28: each upper case for that block

has a determinant-one minor, and its lower case has both algebraic and geometric witnesses.

Depth $m = 16$ corresponds to a complete power-of-two batch of 65,536 points. Here the difference between $t = 4$ and $t = 5$ is operationally concrete: the former certifies every elementary interval of volume 2^{-12} with 16 points, whereas the latter guarantees uniformity only from volume 2^{-11} with 32 points. As ancillary context, a separate authenticated computation using two rank kernels gives the depth-15 distribution alongside the theorem depth in Table 6. This extra row is not part of Theorem 1.1.

TABLE 6. Ancillary exact census at depth 15, alongside the theorem depth 16, for the same 345 pinned pair-aligned windows.

Depth	Exact t -value distribution
15	$t = 3$: 1; $t = 4$: 344
16	$t = 4$: 9; $t = 5$: 336

The nine depth-16 exceptions do not share one universal first obstruction: their recorded first deficient compositions and dependency masks are all distinct, and the number of deficient boundary compositions among these nine windows ranges from 1 to 10. The machine-readable inventory makes that heterogeneity checkable. It would be unjustified to infer a closed-form classification of exceptional starts from these nine cases alone.

The theorem should not be generalized beyond those identifiers. It covers every pair-aligned four-dimensional window of the pinned released table, but not unaligned four-dimensional windows, scrambled points, or modified direction numbers. The ancillary table reports one additional depth but does not broaden the theorem statement. A different source blob is a different theorem input and must be recertified.

Reproduction. The release contains the complete LaTeX source, two separately written standard-library census programs, the frozen 4-by-16 prefix, the full-table census, both block JSON certificates, all 364 minors, captured outputs, and the pinned Lean project. The primary source-fidelity mode additionally accepts a downloaded copy of the complete C++ table and verifies both full hashes before computing. Passing `--census` with that source file recomputes all 345 exact values. The separate `adjacent_depth_census.py` authenticates the same source and recomputes Table 6 with two rank kernels. The default check mode regenerates JSON in memory and byte-compares it with the shipped files without modifying them; an explicit output directory is required to write regenerated artifacts. Correctness checks use explicit runtime exceptions rather than Python `assert`; regression tests confirm that deliberate source corruption is rejected under both ordinary Python and `python -O`. Release checksums are generated only after the final PDF

and archives have been built. The versioned preprint, source, exact certificates, verification software, and Lean companion are available in the Zenodo record at [doi:10.5281/zenodo.21925582](https://doi.org/10.5281/zenodo.21925582).

Declaration of generative-AI use. The author used OpenAI Codex and Anthropic Claude (accessed August 2026) for literature-search assistance, automated analysis and drafting, programming, auditing, and language editing. Claude additionally provided a secondary automated adversarial audit, not independent peer review. Their outputs were non-authoritative suggestions. The named author directed the project and literature sweeps, chose which analyses and prose to retain or revise, reviewed the assembled arguments, code, citations, and manuscript, and approved the release. Neither system is listed as an author.

REFERENCES

- [1] N. Bonneel, D. Coeurjolly, J.-C. Iehl, and V. Ostromoukhov. Sobol’ sequences with guaranteed-quality 2D projections. *ACM Trans. Graph.*, 44(4):97:1–97:16, 2025. [doi:10.1145/3730821](https://doi.org/10.1145/3730821).
- [2] P. Bratley and B. L. Fox. Algorithm 659: Implementing Sobol’s quasirandom sequence generator. *ACM Trans. Math. Software*, 14(1):88–100, 1988. [doi:10.1145/42288.214372](https://doi.org/10.1145/42288.214372).
- [3] J. Dick and F. Pillichshammer. *Digital Nets and Sequences: Discrepancy Theory and Quasi-Monte Carlo Integration*. Cambridge University Press, 2010. [doi:10.1017/CB09780511761188](https://doi.org/10.1017/CB09780511761188).
- [4] S. Joe and F. Y. Kuo. Constructing Sobol’ sequences with better two-dimensional projections. *SIAM J. Sci. Comput.*, 30(5):2635–2654, 2008. [doi:10.1137/070709359](https://doi.org/10.1137/070709359).
- [5] LIRIS Origami. OneTwoSobolSequences: Reference matrices and sampler. GitHub repository, 2025. Commit 163127a0c1731ffe2d43cd685067795c2689a308; accessed 2026-08-14. URL: <https://github.com/liris-origami/OneTwoSobolSequences/tree/163127a0c1731ffe2d43cd685067795c2689a308>.
- [6] P. Marion, M. Godin, and P. L’Ecuyer. An algorithm to compute the t -value of a digital net and of its projections. *J. Comput. Appl. Math.*, 371:112669, 2020. [arXiv:1910.02277](https://arxiv.org/abs/1910.02277), [doi:10.1016/j.cam.2019.112669](https://doi.org/10.1016/j.cam.2019.112669).
- [7] H. Niederreiter. *Random Number Generation and Quasi-Monte Carlo Methods*, volume 63 of *CBMS-NSF Regional Conference Series in Applied Mathematics*. Society for Industrial and Applied Mathematics, Philadelphia, 1992. [doi:10.1137/1.9781611970081](https://doi.org/10.1137/1.9781611970081).
- [8] G. Pirsic and W. C. Schmid. Calculation of the quality parameter of digital nets and application to their construction. *J. Complexity*, 17(4):827–839, 2001. [doi:10.1006/jcom.2001.0597](https://doi.org/10.1006/jcom.2001.0597).
- [9] W. C. Schmid. Projections of digital nets and sequences. *Math. Comput. Simulation*, 55(1–3):239–247, 2001. [doi:10.1016/S0378-4754\(00\)00267-6](https://doi.org/10.1016/S0378-4754(00)00267-6).
- [10] I. M. Sobol’. On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Comput. Math. Math. Phys.*, 7(4):86–112, 1967. [doi:10.1016/0041-5553\(67\)90144-9](https://doi.org/10.1016/0041-5553(67)90144-9).
- [11] X. Wang and I. H. Sloan. Low discrepancy sequences in high dimensions: How well are their projections distributed? *J. Comput. Appl. Math.*, 213(2):366–386, 2008. [doi:10.1016/j.cam.2007.01.005](https://doi.org/10.1016/j.cam.2007.01.005).
- [12] T. Yoshiki. Digital nets on cubature nodes: Inheriting cubature accuracy on low-dimensional projections, 2026. [arXiv:2607.17080](https://arxiv.org/abs/2607.17080), [doi:10.48550/arXiv.2607.17080](https://doi.org/10.48550/arXiv.2607.17080).